

**A FRAMEWORK FOR RISK-AWARE SECURITY
MISCONFIGURATION DETECTION AND
PRIORITIZATION IN CLOUD
INFRASTRUCTURE-AS-CODE ENVIRONMENTS**

Jayathissa E.A.T.N.

258243J

MSc in Computer Science Specialization in Cloud Computing

DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING
Faculty Of Engineering

University of Moratuwa
Sri Lanka

May 2026

**A FRAMEWORK FOR RISK-AWARE SECURITY
MISCONFIGURATION DETECTION AND
PRIORITIZATION IN CLOUD
INFRASTRUCTURE-AS-CODE ENVIRONMENTS**

Jayathissa E.A.T.N.

258243J

Dissertation submitted in partial fulfilment of the requirements for the degree
MSc in Computer Science Specialization in Cloud Computing

DEPARTMENT OF COMPUTER SCIENCE AND
ENGINEERING
Faculty Of Engineering

University of Moratuwa
Sri Lanka

May 2026

DECLARATION

I declare that this is my own work and this Dissertation does not incorporate without acknowledgment any material previously submitted for a Degree or Diploma in any other University or Institute of higher learning and to the best of my knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgment is made in the text. I retain the right to use this content in whole or part in future works (such as articles or books).

Signature:

Date:

The supervisor should certify the Dissertation with the following declaration.

The above candidate has carried out research for the MSc in Computer Science Specialization in Cloud Computing Dissertation under my supervision. I confirm that the declaration made above by the student is true and correct.

Name of Supervisor:

Signature of the Supervisor:

Date:

DEDICATION

This work is dedicated to my academic mentors, colleagues, and peers who have inspired me through their knowledge, curiosity, and commitment to learning. Their guidance, collaboration, and shared pursuit of excellence have been a constant source of motivation throughout this research journey. Furthermore dedicated to all those who supported and inspired me throughout my postgraduate studies.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor for the guidance, valuable feedback, and support provided during this project. The supervisor's academic direction and constructive suggestions helped shape this research into a focused and practically relevant study on cloud security and Infrastructure-as-Code risk prioritization.

I am also thankful to the academic staff of the Department of Computer Science and Engineering, University of Moratuwa, for the knowledge, resources, and academic foundation provided throughout the postgraduate programme. The coursework and discussions in cloud computing, software architecture, security, and research methodology contributed significantly to the development of this report.

I would like to acknowledge my colleagues and peers for their valuable discussions, encouragement, and support throughout this academic journey. Their practical insights into software engineering, cloud deployment, and DevSecOps practices helped refine the motivation and technical direction of this research.

Finally, I extend my heartfelt appreciation to my family for their patience, understanding, and continuous support. Their encouragement enabled me to remain focused and committed throughout the completion of this postgraduate diploma research report.

ABSTRACT

Cloud computing environments increasingly depend on Infrastructure-as-Code (IaC) technologies such as Terraform, Kubernetes manifests, Helm charts, and cloud deployment templates to automate infrastructure provisioning and configuration management. Although IaC improves deployment consistency, repeatability, scalability, and operational efficiency, security misconfigurations remain a major cause of cloud security incidents. Misconfigurations such as overly permissive IAM policies, publicly exposed storage resources, unrestricted network rules, missing encryption settings, and insecure container configurations can introduce serious risks before cloud resources are deployed. Existing IaC security scanners and policy-as-code tools mainly rely on predefined rule-based checks. These tools are effective for identifying known misconfiguration patterns; however, they often generate large volumes of findings without sufficient contextual prioritization. As a result, security and DevOps teams may struggle to identify which findings require urgent remediation and which findings can be treated as lower-priority tasks. This creates alert overload, weak remediation planning, and limited practical usefulness of automated scanning outputs. This research proposes a framework for risk-aware security misconfiguration detection and prioritization in cloud Infrastructure-as-Code environments. The framework combines conventional IaC scanning with contextual risk assessment by considering factors such as scanner severity, public exposure, IAM privilege scope, resource sensitivity, encryption status, and deployment environment. The proposed framework is designed to classify detected findings into prioritized remediation categories such as Critical, High, Medium, and Low. The framework is evaluated using representative Terraform and Kubernetes configuration examples, baseline scanner outputs, and assessment criteria such as detection coverage, prioritization usefulness, alert reduction, and ranking consistency. This research contributes a structured and practical approach for improving cloud IaC security assessment by moving beyond simple rule-based detection toward context-aware remediation support.

Keywords: Cloud Security, Infrastructure-as-Code, Security Misconfiguration, Risk Prioritization, Terraform, Kubernetes, DevSecOps

TABLE OF CONTENTS

Declaration of the Candidate & Supervisor	i
Dedication	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v
List of Figures	vii
List of Tables	viii
1 Introduction	1
1.1 Background	2
1.1.1 Background of Cloud Infrastructure-as-Code	2
1.2 Research Problem	2
1.3 Research Objectives	3
1.4 Research Questions	4
1.5 Scope and Limitations	4
2 LITERATURE REVIEW	5
2.1 Introduction	5
2.2 Cloud Security and DevSecOps Evolution	5
2.3 Infrastructure-as-Code Security	6
2.4 Cloud Security Misconfigurations	6
2.5 Rule-Based IaC Security Scanning	7
2.6 Policy-as-Code and Governance	8
2.7 IAM Risk and Privilege Escalation	9
2.8 Contextual Risk Assessment and Prioritization	9
2.9 Comparative Analysis of Existing Approaches	10
2.10 Research Gap and Summary	11
3 METHODOLOGY	13
3.1 Introduction	13
3.2 Research Design and Approach	13
3.3 Proposed System Architecture	13

3.4	Data Collection and Preparation	14
3.5	Contextual Risk Factor Model	15
3.6	Risk Scoring and Prioritization	15
3.7	Evaluation Methodology	16
3.8	Summary	18
	References	19

LIST OF FIGURES

Figure	Description	Page
Figure 2.1	IaC security lifecycle	7
Figure 2.2	Contextual risk prioritization concept	10
Figure 3.1	Proposed risk-aware IaC security framework architecture	14
Figure 3.2	Risk-aware prioritization workflow	16
Figure 3.3	Evaluation strategy	17

LIST OF TABLES

Table	Description	Page
Table 2.1	Comparison of IaC security detection approaches	8
Table 2.2	Comparison of IaC security detection approaches	12
Table 3.1	Contextual risk factors used in the proposed framework	15
Table 3.2	Risk scoring scale	16
Table 3.3	Evaluation metrics for the proposed framework	17

CHAPTER 1

INTRODUCTION

Cloud computing has become one of the most important foundations of modern digital transformation. Organizations use cloud platforms to provision computing resources rapidly, scale services according to demand, and reduce the operational effort involved in managing physical infrastructure. Cloud adoption has also changed software delivery practices because infrastructure resources can now be created, updated, and destroyed through automated workflows rather than manual configuration alone [1, 2].

In this environment, Infrastructure-as-Code (IaC) has become a central practice for cloud engineering. IaC enables infrastructure to be described through machine-readable configuration files. These files can be version-controlled, reviewed, tested, scanned, and deployed through DevSecOps pipelines. Terraform, Kubernetes manifests, Helm charts, AWS CloudFormation, Azure Resource Manager templates, and similar technologies are commonly used to define networking, compute, storage, identity, and container orchestration resources [1–4].

Although IaC improves consistency and deployment speed, it also shifts security responsibility into infrastructure code. A configuration error written in an IaC template can be reproduced across multiple environments. For example, a single Terraform file can accidentally create a publicly accessible storage bucket, a security group allowing unrestricted inbound access, or an IAM role with excessive permissions. Similarly, a Kubernetes YAML file may define a privileged container, expose an internal service publicly, or omit important pod security controls [5–7].

Therefore, cloud security increasingly depends on the quality of infrastructure definitions before deployment. Traditional security review performed after infrastructure deployment is insufficient because cloud environments change rapidly. DevSecOps practices encourage security checks to be introduced early in the lifecycle, including at code commit, pull request, and CI/CD pipeline stages [8, 9]. This makes IaC security scanning an important pre-deployment control.

However, scanning alone does not solve the problem of cloud misconfiguration. Existing tools such as Checkov, tfsec, and Trivy can identify many known issues, but they may produce a large number of findings with limited contextual prioritization [3, 4, 10]. A finding reported as high severity may not have the same operational impact in all situations. A misconfiguration affecting a public production resource with sensitive data is more urgent than a similar issue in a private test resource. This research is motivated by the need to enrich scanner findings with cloud context and prioritize them according to practical remediation risk.

1.1 Background

1.1.1 Background of Cloud Infrastructure-as-Code

Infrastructure-as-Code treats infrastructure definitions as software artifacts. Instead of manually configuring servers, networks, storage, identity policies, and container platforms, engineers define the desired infrastructure state in configuration files. These files can be stored in source control, reviewed through pull requests, and deployed through automated pipelines. This approach improves repeatability, auditability, and collaboration across development, operations, and security teams.

Terraform is widely used because it provides a declarative approach for provisioning resources across multiple cloud providers. Kubernetes YAML manifests are also important because containerized workloads and microservices are frequently deployed through Kubernetes clusters. CloudFormation, ARM templates, Helm charts, and other IaC technologies contribute to a broader ecosystem where infrastructure, platform configuration, and application deployment become programmable [1, 2].

The security implications of IaC are significant. Infrastructure definitions directly control public exposure, network segmentation, identity permissions, encryption settings, logging, and workload isolation. As a result, insecure IaC can create vulnerabilities even when application code itself is secure. The OWASP Infrastructure as Code Security Cheat Sheet recommends integrating IaC security practices into the software development lifecycle and highlights the need to detect insecure patterns before deployment [5].

Recent empirical research further confirms that IaC does not automatically eliminate security problems. Verdet et al. studied open-source Terraform projects and found that security best-practice adoption varies across cloud providers and resource categories [11, 12]. Their findings support the view that explicit security analysis is still necessary even when infrastructure is scripted. Cloud security also remains affected by common weaknesses such as misconfiguration and poor access control, as emphasized by NSA and CISA guidance on mitigating cloud vulnerabilities [13].

These observations show that IaC security is not only a tooling problem. It is also a prioritization and decision-support problem. Security and DevOps teams need to know which findings should be fixed first, which ones create the highest business and technical risk, and how scanner results should be interpreted in context.

1.2 Research Problem

Current IaC security tools mainly focus on detecting predefined misconfiguration patterns. Rule-based scanners compare infrastructure definitions against built-in or custom checks and report violations. This approach is valuable because it enables early identification of known insecure settings. However, scanner output frequently lacks sufficient context for remediation prioritization.

In practical cloud environments, teams may receive many findings across Terraform files, Kubernetes manifests, and cloud templates. Without contextual prioritization, security teams may struggle to decide which issue should be fixed first. Scanner severity labels alone may not represent operational risk because the impact of a finding depends on public exposure, IAM privilege scope, resource sensitivity, encryption status, and whether the affected resource belongs to production or a non-production environment.

IAM risk is a particularly important example. A simple rule may identify an overly broad permission, but the practical risk depends on whether that permission can be combined with other permissions to create privilege escalation. Hu et al. demonstrate that IAM privilege escalation requires deeper contextual reasoning beyond simple rule matching [14]. Similar contextual reasoning is needed for storage exposure, network access, Kubernetes workload security, and encryption weaknesses.

The research problem can therefore be stated as follows: existing IaC security scanners can detect many misconfigurations, but they provide limited contextual prioritization to support remediation decision-making in cloud environments. A risk-aware framework is required to combine scanner findings with contextual cloud attributes and produce prioritized remediation categories.

1.3 Research Objectives

The main objective of this research is to design and evaluate a framework for risk-aware security misconfiguration detection and prioritization in cloud Infrastructure-as-Code environments.

The specific objectives are:

1. Investigate common security misconfigurations and policy violations in cloud IaC environments.
2. Analyze the limitations of existing rule-based IaC security scanning approaches.
3. Identify contextual risk factors relevant to IaC security prioritization, including scanner severity, public exposure, IAM privilege scope, resource sensitivity, encryption status, and deployment environment.
4. Design a framework that enriches scanner findings with contextual cloud security attributes.
5. Develop a transparent risk-scoring approach to classify findings into Critical, High, Medium, and Low remediation priorities.
6. Evaluate the proposed framework using representative Terraform and Kubernetes configuration examples and compare its output with conventional scanner severity results.

1.4 Research Questions

The research is guided by the following research questions:

1. **What** are the common security misconfiguration patterns found in cloud Infrastructure-as-Code environments?
2. **What** limitations exist in conventional rule-based IaC security scanners when used for remediation prioritization?
3. **Which** contextual cloud security factors are most useful for prioritizing IaC security findings?
4. **How** can a risk-aware scoring model improve the practical usefulness of IaC scanner outputs?
5. **How** does the proposed prioritization approach compare with conventional scanner severity outputs?

1.5 Scope and Limitations

This research focuses on pre-deployment security analysis of cloud Infrastructure-as-Code artifacts. The primary IaC formats considered are Terraform configurations and Kubernetes YAML manifests. The framework also considers cloud template concepts that are relevant to CloudFormation, ARM templates, and Helm charts, although the evaluation is centered on representative Terraform and Kubernetes examples.

The research uses existing tools such as Checkov, tfsec, and Trivy as baseline scanners. The purpose is not to replace these tools but to extend their practical usefulness by applying contextual risk prioritization. The proposed framework considers contextual attributes such as severity, public exposure, IAM privilege scope, resource sensitivity, encryption status, and environment criticality.

The research does not attempt to implement a complete enterprise-grade Cloud Security Posture Management platform. It also does not focus on runtime intrusion detection, malware analysis, or full-scale cloud provider monitoring. The evaluation is limited to representative IaC configurations and controlled examples suitable for a postgraduate diploma research project. Future work can extend the framework to production repositories, multi-cloud environments, and learning-based risk scoring.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter reviews literature and technical practices related to cloud security, Infrastructure as Code, security misconfiguration detection, policy-as-code, IAM risk, and contextual risk prioritization. The purpose of the review is to identify the current state of IaC security and justify the need for a risk-aware prioritization framework.

Cloud security is a broad area covering identity, network security, data protection, workload isolation, monitoring, compliance, and governance. In IaC environments, many of these controls are defined before deployment through configuration files. This makes IaC scanning an important early security practice, but it also introduces the challenge of interpreting scanner findings in a meaningful way.

The literature shows that automated IaC scanning is valuable for identifying known misconfigurations, but it also shows that detection alone is insufficient. Existing tools often provide rule violations and severity labels; however, remediation decisions require a broader understanding of context. This chapter therefore examines both detection approaches and the need for contextual prioritization.

2.2 Cloud Security and DevSecOps Evolution

Cloud security has evolved alongside the growth of cloud-native systems, microservices, containers, and automated delivery pipelines. In traditional environments, infrastructure changes were slower and often controlled by infrastructure teams. In cloud-native systems, infrastructure can be changed frequently by development and platform teams through automated pipelines. This creates speed and flexibility but also increases the risk of rapid propagation of insecure configurations [15–17].

DevSecOps emerged as a response to the need for integrating security into development and operations workflows. Instead of treating security as a final gate, DevSecOps promotes continuous security assessment throughout the software delivery lifecycle. NIST SP 800-218 defines the Secure Software Development Framework as a set of high-level secure development practices that can be integrated into software development processes [8]. OWASP also emphasizes CI/CD pipeline security because pipelines are critical components of modern delivery workflows and attractive targets for attackers [9].

Infrastructure-as-Code fits naturally into DevSecOps because infrastructure definitions are managed as code. IaC files can be scanned during pull requests, code reviews, and pipeline executions. This enables shift-left security, where misconfigurations can be detected before resources are deployed. However, DevSecOps teams often face large volumes of findings from multiple tools, including application scanners, depen-

dency scanners, container scanners, IaC scanners, and cloud posture tools. In this environment, prioritization becomes as important as detection.

The cloud security literature and practitioner guidance consistently identify misconfiguration and poor access control as major cloud risks [13, 18–20]. Therefore, cloud DevSecOps requires security controls that address both configuration correctness and risk-based remediation decision-making. This supports the need for a framework that connects IaC scanning outputs with contextual risk assessment.

2.3 Infrastructure-as-Code Security

IaC security focuses on preventing insecure infrastructure definitions from reaching deployed environments. Since IaC files define resource properties, permissions, networking, and workload settings, static analysis can be used to identify insecure configurations before cloud deployment. This approach is similar to static application security testing, but the analyzed artifacts are infrastructure definitions rather than application source code.

OWASP recommends multiple IaC security practices, including storing IaC in source control, scanning for misconfigurations, protecting secrets, enforcing least privilege, and validating policies before deployment [5]. These practices reflect the growing view that infrastructure code should be subject to the same governance and quality controls as application code.

Empirical research by Verdet et al. provides important evidence for this research. Their study of Terraform projects shows that security best-practice adoption is uneven and that IaC scripting does not automatically prevent misconfigurations, vulnerabilities, or privacy risks [11]. Their later work on security policy adoption across cloud providers further shows that some security guidelines are difficult to translate into concrete configuration policies and that adoption patterns differ across AWS, Azure, and Google Cloud [12].

Figure 2.1 summarizes a typical IaC security lifecycle. The lifecycle begins with infrastructure code creation and continues through scanning, policy checks, context enrichment, remediation, and feedback. The proposed research focuses on the context enrichment and prioritization stages, which are often weaker in conventional scanner workflows.

2.4 Cloud Security Misconfigurations

Cloud security misconfigurations are configuration errors that weaken the security posture of cloud resources. These weaknesses may arise from insecure defaults, misunderstanding of provider controls, excessive permissions, missing encryption, poor network segmentation, or weak logging. NSA and CISA classify misconfiguration and poor access control as major cloud vulnerability categories [13].

IaC Security Lifecycle

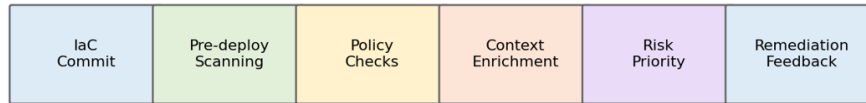


Fig. 2.1: IaC security lifecycle

Common cloud misconfiguration categories include public storage exposure, unrestricted network access, excessive IAM permissions, missing encryption, insecure secrets handling, weak logging and monitoring, and insecure container or Kubernetes settings. In Kubernetes environments, security issues may include privileged containers, hostPath volume use, weak pod security contexts, missing resource limits, or exposed services [6, 7, 21].

Misconfiguration risk is highly contextual. For example, an unencrypted resource containing public test data may be less urgent than a public production database containing sensitive information. Similarly, an open port on an internal testing subnet may not have the same risk as an open management port exposed to the public internet. Therefore, detection needs to be complemented by context-aware risk assessment.

Existing scanner severity levels are useful starting points, but they cannot fully represent cloud-specific impact. Risk depends on exposure, asset sensitivity, identity privilege, environment, and compensating controls. This motivates the use of contextual factors in the proposed framework.

2.5 Rule-Based IaC Security Scanning

Rule-based IaC scanners detect known misconfiguration patterns by comparing infrastructure definitions against predefined checks. Checkov is a static code analysis tool for IaC files and supports Terraform, CloudFormation, Kubernetes, Helm, ARM templates, and other formats [3]. Trivy is a cloud-native security scanner that can identify vulnerabilities, secrets, and IaC misconfigurations across multiple targets [4]. tfsec is a Terraform-focused static analysis scanner designed for local and CI pipeline usage [10].

These tools provide important benefits. They are easy to automate, support developer workflows, and can detect many common misconfiguration patterns before deployment. They also provide machine-readable outputs that can be integrated into

TABLE 2.1: COMPARISON OF IAC SECURITY DETECTION APPROACHES

Approach	Strengths	Limitations	Relevance to this research
Rule-based scanning	Easy to automate; detects known misconfiguration patterns; integrates with CI/CD	Limited context awareness; may generate many findings; depends on predefined checks	Used as baseline detection layer
Policy-as-Code	Converts security and governance requirements into executable policies	Requires accurate policy translation and maintenance; may not rank findings by risk	Supports policy enforcement and governance
CSPM	Provides visibility into deployed cloud posture and compliance state	Often post-deployment; may also create alert overload	Provides broader posture-management perspective
Risk-aware prioritization	Considers exposure, privilege, sensitivity, environment, and severity together	Requires contextual data collection and scoring design	Main contribution of this research

CI/CD systems and reporting dashboards.

However, rule-based scanning has limitations. First, scanners depend on predefined checks and may not identify risks that require broader context. Second, scanners may produce many findings across large repositories, creating alert overload. Third, severity labels may not reflect the real-world risk of a finding in a specific environment. Fourth, different scanners may classify similar findings differently, creating inconsistency.

Table ?? compares major IaC security detection approaches and their relevance to this research.

2.6 Policy-as-Code and Governance

Policy-as-Code (PaC) is the practice of expressing governance, security, and compliance policies as code. PaC enables organizations to automate policy evaluation and reduce reliance on manual review. Open Policy Agent (OPA) is a general-purpose policy engine that supports policy decisions across services, Kubernetes, CI/CD, and other environments [22]. OPA Gatekeeper extends OPA into Kubernetes admission control, allowing policy enforcement before workloads are admitted into a cluster [21, 23]. Policy-as-Code is relevant to IaC security because policies can be applied before infrastructure changes are deployed. For example, an organization may write policies

preventing public storage, unrestricted security group ingress, unencrypted databases, or privileged Kubernetes containers. These policies can be evaluated during code review or pipeline execution.

However, policy-as-code also has challenges. Organizations must correctly translate high-level security requirements into technical rules. Verdet et al. show that not all compliance guidelines can be directly transformed into configuration implementation policies [12]. Policies also require maintenance as cloud provider services change. Furthermore, policy violations still require prioritization because a pipeline may identify many policy failures at once.

Therefore, policy-as-code improves governance but does not remove the need for contextual prioritization. The proposed framework can use policy outputs as part of detection while adding risk scoring to support remediation planning.

2.7 IAM Risk and Privilege Escalation

Identity and Access Management is one of the most critical components of cloud security. IAM controls who can access cloud resources and what actions they can perform. Misconfigured IAM can lead to excessive privilege, unauthorized access, data exposure, and privilege escalation. Cloud IAM is complex because permissions may involve users, roles, groups, service accounts, trust relationships, resource policies, and inherited privileges [24, 25].

Hu et al. propose TAC, an approach for efficient IAM greybox penetration testing. Their work shows that IAM privilege escalation is difficult to detect using simple rule matching because escalation paths may depend on combinations of permissions and relationships [14]. This finding is highly relevant to IaC security because IAM policies and roles are often defined directly in Terraform and cloud deployment templates.

In risk-aware IaC prioritization, IAM privilege scope should be treated as a major risk factor. A misconfiguration affecting a read-only role is usually less severe than a misconfiguration affecting an administrative role. Similarly, a policy allowing role assumption, policy attachment, or permission boundary modification may create higher risk than a narrow permission on a low-sensitivity resource.

The literature therefore supports the inclusion of IAM privilege scope and escalation potential in the proposed framework. This allows the framework to prioritize identity-related findings more accurately than severity-only scanner output.

2.8 Contextual Risk Assessment and Prioritization

Risk assessment evaluates security issues based on likelihood and impact. NIST SP 800-30 provides a general approach for conducting risk assessments, while CVSS provides a standardized scoring system for vulnerabilities [26, 27]. Although IaC misconfigurations are not always CVEs, the idea of combining severity with environmental context is useful for prioritization.

In IaC environments, contextual risk assessment can consider multiple factors. Scanner severity provides the baseline. Public exposure indicates whether the affected resource can be reached from the internet. IAM privilege scope reflects the power of the affected permissions. Resource sensitivity indicates whether the resource stores or processes sensitive data. Environment criticality distinguishes production from development or testing. Encryption status indicates whether data protection controls are missing.

Figure 2.2 illustrates the proposed contextual prioritization concept. Scanner findings are first collected from rule-based tools. These findings are then enriched with contextual attributes. A risk scoring model combines these attributes and produces a prioritized output. This helps security teams focus remediation effort on findings with higher practical risk.

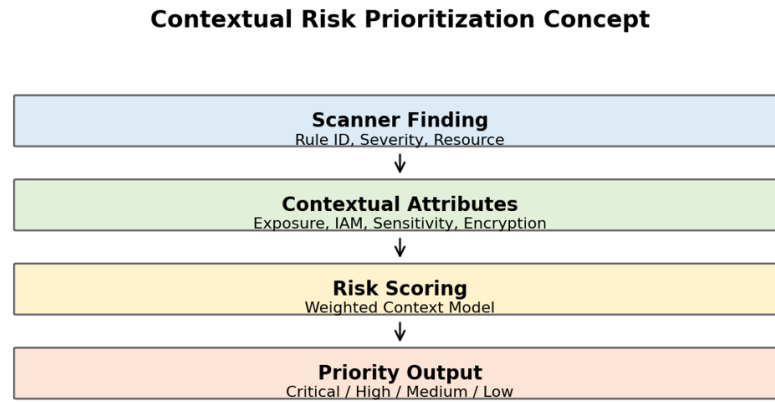


Fig. 2.2: Contextual risk prioritization concept

Contextual prioritization is especially important where security teams face alert overload. A flat list of scanner results may not provide clear remediation order. A risk-aware framework can reduce ambiguity by ranking issues according to impact and urgency. This does not eliminate expert judgment, but it gives practitioners a structured basis for decision-making.

The proposed research uses an explainable weighted scoring model rather than a hidden machine learning model. This is appropriate for a PGDip-level project because it allows each factor and weight to be inspected, justified, and adjusted. Future MSc-level research can extend the framework with historical remediation data or learning-based prioritization.

2.9 Comparative Analysis of Existing Approaches

The reviewed literature and tools show that IaC security has strong detection support but weaker prioritization support. Rule-based scanners identify known misconfigura-

tions, policy-as-code tools enforce organizational rules, and CSPM tools monitor deployed resources. However, these approaches often remain separated from contextual remediation decision support.

Table ?? summarizes the literature synthesis. The synthesis highlights that previous studies and tools support the need for IaC scanning, cloud security governance, and IAM analysis, but they do not fully address unified context-aware prioritization for IaC findings.

2.10 Research Gap and Summary

This chapter reviewed cloud security, DevSecOps, Infrastructure-as-Code security, rule-based scanning, policy-as-code, IAM risk, and contextual prioritization. The review confirms that IaC security is essential because cloud infrastructure definitions directly influence public exposure, identity permissions, encryption, network access, and workload security.

The main research gap identified is that existing IaC security approaches detect known misconfigurations but provide limited contextual prioritization of findings based on public exposure, IAM privilege scope, resource sensitivity, encryption status, and deployment environment. Existing tools are valuable for detection, but their outputs need to be enriched for better remediation decision-making.

This gap leads to the proposed methodology in Chapter 3. The proposed framework combines baseline scanner findings with contextual risk factors and produces prioritized remediation categories. In this way, the research moves beyond simple detection toward practical risk-aware cloud security decision support.

TABLE 2.2: COMPARISON OF IAC SECURITY DETECTION APPROACHES

Source / Area	Main contribution	Limitation / gap	Connection to this research
Terraform security practices [11]	Empirical analysis of security best-practice adoption in Terraform projects	Focuses on adoption patterns rather than prioritization	Supports need for IaC security analysis
Terraform policy adoption [12]	Shows provider differences and policy translation challenges	Does not propose unified risk scoring	Supports contextual interpretation
IAM greybox testing [14]	Shows IAM privilege escalation requires deeper analysis	Focuses on IAM rather than all IaC findings	Supports IAM privilege as risk factor
NIST SSDF [8]	Provides secure development practices	General framework, not IaC-specific prioritization	Supports shift-left security
NSA/CISA cloud guidance [13]	Identifies misconfiguration and poor access control as cloud risks	Guidance-level document	Supports selected risk factors
OWASP IaC guidance [5]	Practical IaC security recommendations	Does not provide scoring model	Supports pre-deployment security checks
Checkov, Trivy, tfsec [3, 10]	Detect known IaC misconfiguration patterns	Rule-based outputs may lack context	Used as baseline scanner layer
OPA/Gatekeeper [22, 23]	Enables policy-as-code enforcement	Policy failures still require prioritization	Supports governance layer

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter presents the methodology used to design and evaluate the proposed risk-aware security misconfiguration detection and prioritization framework for cloud Infrastructure-as-Code environments. The methodology is designed to transform raw scanner findings into contextually prioritized remediation outputs.

The proposed framework follows a layered approach. First, IaC artifacts are collected and prepared. Second, baseline scanners are used to detect misconfiguration findings. Third, contextual attributes are extracted from the findings and related resource definitions. Fourth, a risk-scoring model calculates a priority score. Finally, the output is compared against baseline scanner severity results to evaluate the practical usefulness of the framework.

3.2 Research Design and Approach

This study follows a design-oriented research approach because the main objective is to create and evaluate a technical artifact. The artifact is a framework that improves IaC security assessment by adding contextual prioritization. The research problem is practical and technical: conventional scanners detect misconfigurations, but practitioners require better decision support to prioritize remediation.

The methodology includes problem identification, artifact design, framework development, demonstration using representative IaC examples, and evaluation using defined metrics. This structure is suitable for a postgraduate diploma project because it provides a clear pathway from literature gap to proposed solution and evaluation. The framework is designed to be tool-agnostic. It can accept scanner findings from Checkov, tfsec, or Trivy, and it can be extended to other scanners in future work. The current evaluation focuses on Terraform and Kubernetes because these represent common cloud infrastructure and container orchestration configuration formats.

3.3 Proposed System Architecture

The proposed framework consists of five main layers: IaC input layer, security scanning layer, context extraction layer, risk scoring layer, and reporting layer. Figure 3.1 shows the overall architecture.

The IaC input layer accepts infrastructure definitions such as Terraform files and Kubernetes YAML manifests. These artifacts may define cloud storage, network rules, IAM roles, compute resources, container workloads, and service exposure. The scanning layer runs existing security scanners and produces baseline findings. The context

extraction layer enriches each finding with attributes such as public exposure, IAM privilege scope, resource sensitivity, encryption status, and deployment environment. The risk scoring layer combines these attributes into a final score. The reporting layer ranks findings and provides remediation priority categories.

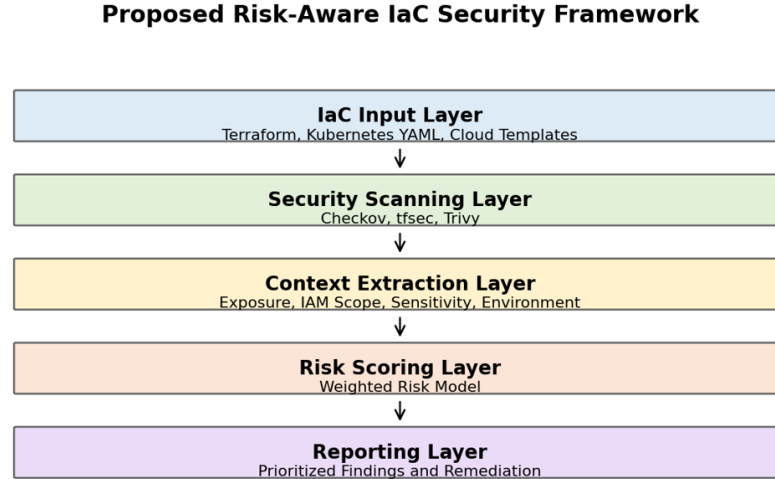


Fig. 3.1: Proposed risk-aware IaC security framework architecture

This architecture intentionally reuses existing scanners rather than replacing them. The contribution is the prioritization layer that improves interpretation of scanner results. This makes the framework practical because organizations can adopt it alongside their existing DevSecOps tools.

3.4 Data Collection and Preparation

The data used in this research consists of IaC configuration files and scanner-generated findings. Terraform files and Kubernetes YAML manifests will be collected from open-source examples, benchmark repositories, and controlled insecure configurations created for testing. The selected examples will cover common cloud resource categories such as storage, networking, IAM, compute, and container workloads.

Data preparation involves organizing IaC artifacts into test cases. Each test case contains one or more IaC files and expected security-relevant properties. For example, a storage test case may include public access settings and encryption settings. A Kubernetes test case may include container privilege settings, service exposure, or security context configuration. An IAM test case may include broad permissions or risky trust relationships.

Baseline scanners will be executed against each test case. The scanner outputs will be parsed to extract issue identifiers, descriptions, severity labels, affected resources, file paths, and remediation suggestions. These fields form the initial finding dataset. Additional contextual attributes are then manually or semi-automatically annotated based on the resource definitions and test-case metadata.

3.5 Contextual Risk Factor Model

The risk factor model defines the attributes used to prioritize scanner findings. The selected factors are based on the literature review, cloud security guidance, and practical cloud risk considerations. Table 3.1 summarizes the factors used in the proposed framework.

The model includes scanner severity as a baseline but does not rely on it alone. Public exposure is included because internet-accessible resources increase attack surface. IAM privilege scope is included because excessive permissions can lead to unauthorized access or privilege escalation. Resource sensitivity is included because the impact of a finding depends on the type and value of the affected asset. Environment criticality is included because production resources generally require higher priority. Encryption risk is included because missing data protection controls can increase confidentiality and compliance impact.

TABLE 3.1: CONTEXTUAL RISK FACTORS USED IN THE PROPOSED FRAMEWORK

Risk factor	Description	Example scoring range
Scanner severity	Baseline severity assigned by the IaC scanner	1-5
Public exposure	Whether the affected resource is reachable from the public internet	0-5
IAM privilege scope	Degree of sensitive or administrative permission involved	0-5
Resource sensitivity	Business or data sensitivity of the affected resource	0-5
Environment criticality	Whether the resource is production, staging, or development	0-5
Encryption risk	Whether required encryption or data protection controls are missing	0-3

3.6 Risk Scoring and Prioritization

The framework uses a transparent weighted additive scoring model. The purpose of the model is to provide an explainable priority score rather than a black-box prediction. The initial score is calculated as follows: Risk Score = Severity + Exposure + Privilege + Sensitivity + Criticality + Encryption Risk

Each factor is assigned a score based on predefined criteria. The total score is mapped into remediation categories. Table 3.2 shows the proposed mapping. The

thresholds can be adjusted during evaluation based on observed behavior and supervisor feedback.

TABLE 3.2: RISK SCORING SCALE

Priority level	Score range	Interpretation	Example remediation action
Critical	22 and above	High likelihood or impact due to multiple severe contextual factors	Block deployment and remediate immediately
High	16-21	Important risk requiring near-term remediation	Fix before production release or require approval
Medium	9-15	Moderate risk that should be tracked and scheduled	Remediate in normal sprint/security backlog
Low	Below 9	Low contextual risk or informational finding	Monitor, document, or fix when convenient

Figure 3.2 illustrates the workflow for risk-aware prioritization. The process begins with IaC collection and scanning. The scanner output is parsed and enriched with context. The scoring engine calculates the final risk score and ranks findings according to remediation priority.

Risk-Aware Prioritization Workflow



Fig. 3.2: Risk-aware prioritization workflow

3.7 Evaluation Methodology

The evaluation aims to determine whether the proposed framework improves the usefulness of scanner outputs for remediation decision-making. Since the framework is not designed to replace scanners, the evaluation focuses on prioritization rather than detection accuracy alone.

The baseline is the raw scanner output produced by Checkov, tfsec, or Trivy. The framework output is the prioritized list after contextual risk scoring. The comparison will examine whether contextually important findings are ranked higher than less important findings and whether the framework reduces ambiguity in remediation planning. The main evaluation metrics are detection coverage, prioritization usefulness, alert reduction, ranking consistency, and baseline comparison. Table 3.3 summarizes these metrics.

TABLE 3.3: Evaluation metrics for the proposed framework

Metric	Purpose	Assessment method
Detection coverage	Measures how many baseline findings are captured and processed	Compare number of scanner findings against processed framework findings
Prioritization usefulness	Assesses whether high-impact contextual findings are ranked higher	Manual review of ranked output against defined risk criteria
Alert reduction	Measures whether low-risk findings can be prioritized	Compare number of findings marked Critical/High before and after context scoring
Ranking consistency	Checks whether similar cases receive similar priority levels	Review repeated or similar test cases
Baseline comparison	Compares scanner severity-only output with framework priority output	Identify changes in rank and justify with contextual factors

Evaluation Strategy

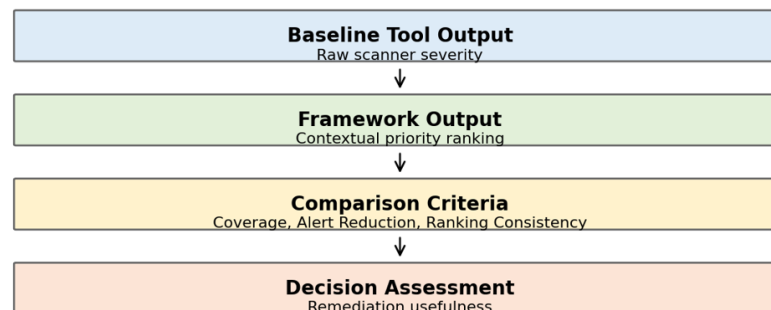


Fig. 3.3: Evaluation strategy

3.8 Summary

This chapter presented the methodology for designing and evaluating the proposed risk-aware IaC security prioritization framework. The methodology follows a layered structure that begins with IaC artifact collection and baseline scanning, continues through contextual enrichment and risk scoring, and ends with prioritized reporting and evaluation. The proposed framework is practical because it builds on existing scanners rather than replacing them. It is also explainable because the scoring model uses visible contextual factors and transparent thresholds. The evaluation strategy focuses on practical remediation usefulness, which aligns with the research problem identified in the literature review. Overall, the methodology establishes a clear pathway for improving IaC security analysis by converting raw scanner findings into prioritized remediation decisions. This supports the broader goal of improving cloud DevSecOps workflows and reducing security risk caused by unresolved misconfigurations.

REFERENCES

- [1] R. Zhang, M. Smythe, C. Hooper, T. Hinrichs, L. Evenson, and T. Sandall, “Opa gatekeeper: Policy and governance for kubernetes,” *Kubernetes Blog*, 2019.
- [2] K. Satirli and T. Dolezal, *Terraform Cookbook*. " O'Reilly Media, Inc.", 2024.
- [3] C. Wang, D. L. Martinez, and E. Cole, “Ai-assisted detection and remediation of cis benchmark violations in kubernetes worker nodes,” 2024.
- [4] I. Eleweke, “Automating trust at scale: Infrastructure-as-code for secure and compliant ai environments in the us,” 2025.
- [5] O. C. S. Series, “Infrastructure as code security cheatsheet,” 2021.
- [6] T. OWASP, “Web application security risks. 2021,” 10.
- [7] M. Souppaya, J. Morello, and K. Scarfone, “Application container security guide,” National Institute of Standards and Technology, Tech. Rep., 2017.
- [8] M. Souppaya, K. Scarfone, and D. Dodson, “Secure software development framework (ssdf) version 1.1,” *NIST Special Publication*, vol. 800, no. 218, pp. 800–218, 2022.
- [9] V. P. Syroid, “Information system for integrating security tools into ci/cd pipelines,” 2025.
- [10] A. Ahmad, “Operationalizing owasp asvs level 1 in ci/cd: Reproducible security testing using one open source and one industrial case study,” 2025.
- [11] A. Verdet, *Exploring security practices in infrastructure as code: An empirical study*. Ecole Polytechnique, Montreal (Canada), 2023.
- [12] A. Verdet, M. Hamdaqa, L. D. Silva, and F. Khomh, “Assessing the adoption of security policies by developers in terraform across different cloud providers,” *Empirical Software Engineering*, vol. 30, no. 3, p. 74, 2025.
- [13] T. Dietz, “When nations attack the cloud: Understanding cyber threats to modern infrastructure,” *Available at SSRN 5795402*, 2025.
- [14] Y. Hu, W. Wang, S. Khurshid, and M. Tiwari, “Interactive greybox penetration testing for cloud access control using iam modeling and deep reinforcement learning,” *arXiv preprint arXiv:2304.14540*, 2023.
- [15] P. L. Foalem, F. Khomh, L. Da Silva, and E. Merlo, “An empirical study of policy-as-code adoption in open-source software projects,” *arXiv preprint arXiv:2601.05555*, 2026.

- [16] J. Humble and D. Farley, *Continuous delivery: reliable software releases through build, test, and deployment automation*. Pearson Education, 2010.
- [17] G. Kim, J. Humble, P. Debois, J. Willis, and N. Forsgren, *The DevOps handbook: How to create world-class agility, reliability, & security in technology organizations*. It Revolution, 2021.
- [18] K. Authors, “Kubernetes documentation, 2024,” URL: <https://kubernetes.io/docs/> (: 29 2024 .), 2024.
- [19] P. H. Leocadio, “Aws master class chapter 14: Aws well-architected framework,” *Authorea Preprints*, 2025.
- [20] P. Sharma and N. Nellutla, “Automated compliance-aware devsecops for ai-driven iot data streams using cloud-native ml across aws, gcp, and azure,” in *2025 20th International Joint Symposium on Artificial Intelligence and Natural Language Processing (ISAI-NLP)*. IEEE, 2025, pp. 1–6.
- [21] D. Mehta, “The evolution of kubernetes: From container orchestrator to universal control plane a comprehensive survey of production experiences, architecture patterns, and decision frameworks,” 2026.
- [22] R. Singh, A. Yeboah-Ofori, S. Kumar, and A. Ganiyu, “Fortifying cloud devsecops security using terraform infrastructure as code analysis tools,” in *2024 International Conference on Electrical and Computer Engineering Researches (ICE-CER)*. IEEE, 2024, pp. 1–6.
- [23] O. P. Agent, “Open policy agent documentation,” *Letzter Zugriff*, vol. 3, p. 2024, 2024.
- [24] D. Holkuziev, “Infrastructure automation in cloud environments using terraform,” *Universal Library of Engineering Technology*, vol. 2, no. 3, 2025.
- [25] L. A. Barnett, L. D’Antoni, A. Goel, R. G. Kici, N. Rungta, M. Southern, and C. Sung, “Modeling the aws authorization engine,” in *2025 Formal Methods in Computer-Aided Design (FMCAD)*. IEEE, 2025, pp. 1–10.
- [26] S. Kivioja, “Improving software vulnerability management with epss,” *Information Technology*, 2024.
- [27] B. Schooley, D. Hansen, E. J. Richmond, M. J. Canite, and N. M. Huaira Reyna, “Visualizing cybersecurity diagrams: An empirical analysis of common weakness enumeration images,” in *International Conference on Human-Computer Interaction*. Springer, 2024, pp. 296–317.

- [28] L. Karadsheh, J. E. Barnard, M. Arafah, and A. F. Shubita, “Generative ai and the evolving threat landscape: A security professional’s guide,” in *Examining Cybersecurity Risks Produced by Generative AI*. IGI Global Scientific Publishing, 2025, pp. 71–98.
- [29] A. J. Adelusi, “Securing multi-cloud environments in government and enterprise systems: Challenges, frameworks, and best practices,” *International Journal of Technology, Management and Humanities*, vol. 12, no. 01, pp. 120–132, 2026.
- [30] L. P. T. Ferreira, “Cloud security risk and readiness,” Master’s thesis, Universidade de Lisboa (Portugal), 2017.
- [31] M. Arafat, “Information security management system challenges within a cloud computing environment,” in *Proceedings of the 2nd International Conference on Future Networks and Distributed Systems*, 2018, pp. 1–6.
- [32] J. T. Force, “Security and privacy controls for information systems and organizations,” National Institute of Standards and Technology, Tech. Rep., 2020.
- [33] H. Nerella and P. S. Puvvada, “Reinforcing kubernetes security: A gap analysis and modern security practices for enterprise,” in *2024 First International Conference on Data, Computation and Communication (ICDCC)*. IEEE, 2024, pp. 780–785.
- [34] J. Cook, “The dockerfile,” in *Docker for Data Science: Building Scalable and Extensible Data Infrastructure Around the Jupyter Notebook Server*. Springer, 2017, pp. 81–101.
- [35] M. Tamanna, S. Hamer, M. Tran, S. Fahl, Y. Acar, and L. Williams, “Unraveling challenges with supply-chain levels for software artifacts (slsa) for securing the software supply chain,” *Available at SSRN 5119626*, 2024.
- [36] J. Girhotra and A. Byrisetty, “Securing cloud-native applications (cnas): A case study of practices in a large it company,” 2025.
- [37] W. Merry Sr, “Virtual networking cybersecurity and vulnerabilities in cloud computing applications: a systematic review,” 2023.
- [38] J. Broad, *Risk Management Framework: A lab-based approach to securing Information Systems*. Newnes, 2013.
- [39] M. Sahinoglu, “Cyber security risk assessment and optimal risk management of a national vulnerability database,” *International Journal of Computer Theory and Engineering*, vol. 16, no. 4, pp. 104–126, 2024.
- [40] S. Sudarsan, M. Agrawal, and A. Banerjee, “Ai-assisted detection of malicious changes in infrastructure-as-code for secure devops,” in *2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC)*. IEEE, 2026, pp. 1–6.

- [41] N. Tallapaneni and S. Yerukula, “End-to-end security automation for aws data pipelines: Leveraging python and iam policy intelligence,” *Available at SSRN 5757042*, 2025.
- [42] M. M. I. Jim, “Cloud security posture management automating risk identification and response in cloud infrastructures,” *Academic Journal on Science, Technology, Engineering & Mathematics Education*, vol. 4, no. 3, pp. 10–69 593, 2024.
- [43] S. Nagasundari, P. Manja, P. Mathur, and P. B. Honnavalli, “Extensive review of threat models for devsecops,” *IEEE Access*, 2025.
- [44] R. K. Jayalath, H. Ahmad, D. Goel, M. S. Syed, and F. Ullah, “Microservice vulnerability analysis: A literature review with empirical insights,” *IEEE Access*, vol. 12, pp. 155 168–155 204, 2024.
- [45] J. Y. Zhang and Y. Zhang, “Quantitative devsecops metrics for cloud-based web microservices,” *IEEE Access*, vol. 12, pp. 160 317–160 342, 2024.
- [46] A. Mittal, “Ai-augmented devsecops pipelines for secure and scalable service-oriented architectures in cloud-native systems,” in *2025 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. IEEE, 2025, pp. 79–84.
- [47] N. Dinh, V. T. Hoang, B. N. Van, T. H. Huong, H. D. T. Hong, H. N. Trung, and K. T. Trung, “Enhancing smart contract security through devsecops: An adaptive approach for vulnerability detection,” *IEEE Access*, 2025.